

ELL 409 Course Project 1: Regression & Forecasting Under Distribution Shift

Group 9

Nishi Yadav (2023MT10388)
Tushar Kumar Sinha (2023MT60608)
Videep Ekbote (2023MT10940)
Yukti Yadav (2023MT10387)

Abstract

This report provides a detailed account of our end-to-end methodology for addressing the four sub-problems (Parts A–D) in the ELL 409 Course Project 1. Each part poses a distinct modelling challenge: correlated linear regression, localised nonlinear interactions, panel forecasting with lag features, and forecasting under distribution shift with outlier contamination. We adopt a *part-specific* modelling strategy built entirely upon classical closed-form Ridge Regression, varying only the feature engineering and preprocessing per part: standard Ridge for Part A, Polynomial Ridge with degree-6 expansion for Part B, Ridge with standard normalisation for Part C, and Ridge with extensive lag-derived feature engineering, entity-mean target encoding, and robust normalisation for Part D. Preprocessing pipelines include median imputation, one-hot encoding, standard and robust normalisation, and outlier clipping. A strict validation protocol using `valid.csv` governs all hyperparameter choices. We present a global OLS baseline, derive the mathematics of every model, justify each design decision (including why certain models such as IRLS, Lasso, and kernel methods were *not* adopted), provide a part-wise error breakdown, and include an ablation study on engineered y-lag features for Parts C and D.

Contents

1	Exploratory Data Analysis and Preprocessing	4
1.1	Key EDA Observations	4
1.2	Preprocessing Pipeline	4
1.2.1	Categorical Encoding: One-Hot Encoding	5
1.2.2	Missing Value Imputation: Median Imputation	5
1.2.3	Feature Scaling: Standard vs. Robust Normalisation	6
1.2.4	Entity-Mean Target Encoding (Part D Only)	6
1.2.5	Sorting for Panel Data (Parts C, D)	7
1.2.6	Feature Matrix Construction	7
2	Validation Protocol	7
3	Baseline Model	8
4	Part-Specific Models: Methodology and Mathematics	8
4.1	Part A: Ridge Regression for Correlated Linear Signals	8
4.1.1	Intuition and Model Selection	8
4.1.2	Mathematical Formulation	8
4.1.3	Why Ridge and not Lasso?	9
4.1.4	Preprocessing (Part A)	9
4.1.5	Hyperparameter Tuning	9
4.2	Part B: Polynomial Ridge Regression for Nonlinear Interactions	9
4.2.1	Intuition and Model Selection	9
4.2.2	Feature Expansion $\Phi(X)$	9
4.2.3	Mathematical Formulation	10
4.2.4	Preprocessing (Part B)	10
4.3	Part C: Ridge Regression with Lag Features for Panel Forecasting	10
4.3.1	Intuition and Model Selection	10
4.3.2	Mathematical Formulation	10
4.3.3	Preprocessing (Part C)	11
4.3.4	Hyperparameter Tuning	11
4.4	Part D: Ridge Regression with Lag-Derived Feature Engineering	11
4.4.1	Intuition and Model Selection	11
4.4.2	Y-Lag Feature Engineering	11
4.4.3	Entity-Mean Target Encoding	12
4.4.4	Mathematical Formulation	12
4.4.5	Preprocessing (Part D)	13
4.4.6	Hyperparameter Tuning	13
5	Models Considered but Not Adopted	13
5.1	IRLS Robust Ridge Regression (Part D)	13
5.2	Lasso Regression (L_1 Penalty)	14
5.3	Elastic Net	14
5.4	Kernel Ridge Regression / Support Vector Regression	14
5.5	Polynomial Features for Parts C/D	15
5.6	Why Only <code>numpy</code> and <code>pandas</code> ?	15

6	Ablation Study: The Impact of Lag Features	15
7	Part-Wise Error Breakdown and Results	16
8	Discussion	17
8.1	Why Part-Specific Models?	17
8.2	Why Ridge Over All Other Models?	17
8.3	Strengths of the Approach	18
8.4	Limitations and Potential Improvements	18
9	Conclusion	18

1 Exploratory Data Analysis and Preprocessing

1.1 Key EDA Observations

Before building any model we performed thorough exploratory analysis on `train.csv`. The principal findings are:

1. **Multicollinearity in Part A.** Several pairs among `x1`–`x20` exhibit strong Pearson correlations, notably `x2` and `x10` ($|\rho| \approx 0.9998$) and `x1` and `x3` ($|\rho| \approx 0.93$). This makes the Gram matrix $X^\top X$ near-singular, causing unregularised OLS to produce unstable weights with extreme variance.
2. **Nonlinear structure in Part B.** While a few features such as `x1` and `x3` exhibit moderate-to-strong linear correlation with y ($|\rho| \approx 0.78$ and 0.84 respectively), a purely linear model achieves only $R^2 \approx 0.74$, leaving substantial variance unexplained. Analysis of `x1` across quantile buckets reveals that y remains near zero in the central region of the feature distribution but rises sharply to mean values of ± 42 in the tails, a smooth nonlinear effect that a linear model cannot fully capture.
3. **Feature distributions.** The continuous covariates `x1`–`x20` are approximately normally distributed, with near-zero skewness across all features (maximum $|\text{skew}| = 0.16$ for `x3`). The discrete covariates `z1`–`z8` take integer values in $\{0, \dots, 9\}$ and are statistically consistent with a uniform distribution (chi-square test: $p > 0.13$ for all eight features). Notably, `x2` and `x10` exhibit slightly elevated kurtosis (≈ 1.88), consistent with their near-perfect correlation ($|\rho| \approx 0.9998$), they are effectively the same feature.
4. **Temporal dependence in Parts C/D.** The lag features `y_lag1` and `y_lag7` are strongly correlated with the target y in Part C (Pearson $\rho \approx 0.97$ and $\rho \approx 0.91$ respectively), confirming strong autoregressive structure. Missing rates for these features are low in Part C (`y_lag1`: 0.83%, `y_lag7`: 5.83%) and zero in Part D, so both lag features are treated as regular input features after median imputation of missing entries.
5. **Distribution shift in Part D.** Part D covers timestamp range 120–179, extending beyond the Part C training range of 0–119. Comparing the training distributions of Parts C and D reveals covariate shift in several features, notably `x3` (mean shift of 0.40) and `x14` (mean shift of 0.30). The target y also shifts slightly in mean (+0.36 in Part C vs -1.01 in Part D) with a marginally higher standard deviation (10.81 vs 11.52).
6. **Missing-value patterns.** Features `m1`–`m6` have missing rates ranging from 5.63% (`m2`) to 14.00% (`m1`). All missing values are imputed using column-wise training medians before model fitting.
7. **Categorical cardinality.** `dept` has 25 unique levels, `building_type` has 6, and `sensor_class` has 4. For Parts C and D, `entity_id` is a high-cardinality categorical: the training set contains 420 unique entities, while the validation and test sets each contain 90 entities. Additionally, `dow` takes 7 values $\{0, \dots, 6\}$ and `month_phase` takes 4 values $\{1, 2, 3, 4\}$.

1.2 Preprocessing Pipeline

Based on the EDA findings above, we designed a part-specific preprocessing pipeline. All statistics (medians, means, standard deviations, MADs, unique categories) are fitted

exclusively on training data and then applied to validation/test sets to avoid information leakage.

1.2.1 Categorical Encoding: One-Hot Encoding

Discrete and categorical features (`dow`, `month_phase`, `dept`, `building_type`, `sensor_class`, and additionally `entity_id` and `holiday_flag` for Parts C/D) must be converted into a numerical representation.

We chose **one-hot encoding** (OHE). Given a categorical column C with k unique training levels $\{c_1, \dots, c_k\}$, we create k binary indicator features:

$$I_{c_j}(x) = \begin{cases} 1 & \text{if } C(x) = c_j, \\ 0 & \text{otherwise.} \end{cases} \quad (1)$$

Why OHE over ordinal encoding? Ordinal encoding assigns integer codes $0, 1, \dots, k-1$ to categories. A linear model then assumes a monotonic relationship between the code and the target, e.g., that `dept = 24` has $24\times$ the effect of `dept = 1`, which is semantically meaningless. OHE avoids this by treating each level as an independent binary indicator.

Handling unseen categories. The unique levels are extracted from the training set alone. If validation or test data contains a category not seen during training, its one-hot vector is all-zero, gracefully yielding no contribution. Missing one-hot columns in the validation/test matrices are filled with zeros.

Implementation detail. For Parts A and B (static rows), we encode [`dow`, `month_phase`, `dept`, `sensor_class`, `building_type`]. For Parts C and D (panel rows), we additionally encode `entity_id` and `holiday_flag`, because (i) `entity_id` acts as a grouping factor whose fixed effects are important for per-entity baselines, and (ii) `holiday_flag` is a binary variable already but is included in the encoding pipeline for consistency.

1.2.2 Missing Value Imputation: Median Imputation

Features `m1`–`m6` contain NaN entries. We impute these using the **column-wise training median**.

Why median over mean? The sample mean $\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$ is highly sensitive to outliers; a single extreme value can shift it arbitrarily. The sample median $\tilde{x}$ has a *breakdown point* of 50%, meaning up to half the data can be corrupted before the statistic becomes unreliable. Given that `m1`–`m6` have missing rates up to 14% and the data contains outliers in some parts, median imputation provides a more robust and representative fill value than the mean.

Protocol. Let $\tilde{m}_j$ be the median of column m_j computed on `train.csv`. For every row i where $m_{i,j}$ is missing, we set $m_{i,j} \leftarrow \tilde{m}_j$. The same $\tilde{m}_j$ values are applied to `valid.csv` and `test.csv`.

1.2.3 Feature Scaling: Standard vs. Robust Normalisation

Scaling is essential because: (i) features with larger magnitudes dominate the L_2 penalty in Ridge regression, biasing the solution; (ii) the condition number of $X^\top X$ directly affects numerical stability of the closed-form solve.

Standard Z-score normalisation (Parts A, B, C). For each continuous column j we compute training-set statistics $\mu_j = \frac{1}{n} \sum_i x_{ij}$ and $\sigma_j = \sqrt{\frac{1}{n-1} \sum_i (x_{ij} - \mu_j)^2}$, and transform:

$$x'_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}. \quad (2)$$

If $\sigma_j < 10^{-8}$ (near-constant column), we set the column to zero. This is used for Parts A, B, and C. For Parts A and B, the marginal distributions are approximately symmetric with moderate tails. For Part C, standard scaling is sufficient because the panel covariates are well-behaved and robust normalisation provided negligible improvement on validation MAE.

Robust normalisation (Part D). For Part D, we use the **Median Absolute Deviation (MAD)** as the scale estimator:

$$\text{MAD}_j = \text{median}(|x_{ij} - \tilde{x}_j|), \quad \hat{\sigma}_j = 1.4826 \times \text{MAD}_j, \quad (3)$$

where the constant 1.4826 makes $\hat{\sigma}_j$ a consistent estimator of σ under normality. The transformation is then:

$$x'_{ij} = \frac{x_{ij} - \tilde{x}_j}{\hat{\sigma}_j}. \quad (4)$$

Additionally, we clip values outside $[\tilde{x}_j - 3.5\hat{\sigma}_j, \tilde{x}_j + 3.5\hat{\sigma}_j]$ to the boundary values. This prevents a small number of extreme observations from distorting the normalisation or introducing numerical instability.

Part D uses this MAD-based robust normalisation for all base continuous features (`x1-x20`, `z1-z8`, `m1-m6`, `timestamp`, `y_lag1`, `y_lag7`). Additionally, Part D introduces 11 engineered lag features (see Section 4.4). These derived features, which include products, squares, and logarithms of lags, can have extreme magnitudes. We therefore apply a separate post-processing step:

1. **Clip** each engineered column to $[\mu_j - 3\sigma_j, \mu_j + 3\sigma_j]$.
2. **Standard Z-score normalise** the clipped values.

This two-stage strategy ensures that the high-variance engineered features do not dominate the L_2 penalty in Ridge regression.

1.2.4 Entity-Mean Target Encoding (Part D Only)

Part D suffers from covariate shift: the training and test distributions of X differ, i.e. $P_{\text{train}}(X) \neq P_{\text{test}}(X)$, even though the conditional $P(Y|X)$ may be approximately preserved. Different entities have different baseline activity levels.

To capture entity-level baselines, we engineered an additional feature:

$$\text{entity_mean_}y_e = \frac{1}{|\mathcal{T}_e|} \sum_{t \in \mathcal{T}_e} y_{e,t}, \quad (5)$$

where $\mathcal{T}_e$ is the set of training timestamps for entity e . For entities unseen at training time, we fall back to the global training mean. This feature acts as an entity-specific intercept, helping the model adjust predictions when entity baselines shift.

1.2.5 Sorting for Panel Data (Parts C, D)

All panel datasets are sorted by (`entity_id`, `timestamp`) before any processing. Although the task is row-level prediction (not recursive forecasting), sorting ensures:

- Consistent alignment of indices when mapping predictions back to `test.csv`.
- Correct association of lag features to their corresponding rows.
- Reproducible preprocessing when constructing the design matrix.

1.2.6 Feature Matrix Construction

After preprocessing, the design matrix is assembled from all columns except metadata (`row_id`, `mode`, `part`, `entity_id`, `timestamp`, `y`). Remaining NaN entries are filled with zero. For Parts A, C, and D, a leading column of ones (bias/intercept) is prepended directly to the feature matrix. For Part B, the bias column is added *after* polynomial expansion to avoid polluting the expansion with the constant feature:

$$X_{\text{bias}} = [\mathbf{1} \ X] \in \mathbb{R}^{n \times (d+1)}. \quad (6)$$

2 Validation Protocol

A rigorous validation protocol is critical to avoid overfitting hyperparameters to noise. We followed these steps:

1. **Fit preprocessing on train.csv only.** All imputers (medians), scalers (mean/std or median/MAD), one-hot category lists, and clipping bounds are computed from the training set.
2. **Apply frozen pipeline to valid.csv.** The validation set is transformed using the same fitted parameters, no statistics are recomputed.
3. **Hyperparameter selection on validation MAE.** For each part, we perform a search over the regularisation parameter λ . The configuration yielding the lowest MAE on `valid.csv` is selected.
4. **Report part-wise validation performance.** We record MAE_A , MAE_B , MAE_C , MAE_D on the validation set with the selected hyperparameters.
5. **Retrain on train + valid for final submission.** After freezing all hyperparameters, we merge `train.csv` and `valid.csv`, refit the entire preprocessing pipeline on this combined set, retrain the model, and generate predictions on `test.csv`.

This protocol strictly prevents data leakage at every stage.

3 Baseline Model

To contextualise the gains from our part-specific models, we first trained a **global unregularised Ordinary Least Squares (OLS)** model on all training rows without distinguishing parts.

The OLS estimator minimises the squared loss:

$$\hat{w}_{\text{OLS}} = \arg \min_w \|y - Xw\|_2^2 \quad (7)$$

$$\hat{w}_{\text{OLS}} = (X^\top X)^{-1} X^\top y \quad (8)$$

Baseline observations.

- On Part A, OLS achieves reasonable MAE because the signal is mostly linear, but the near-singular $X^\top X$ (multicollinearity) produces unreliable, large-magnitude weights.
- On Part B, OLS *severely* underfits because it cannot capture the localised nonlinear interactions.
- On Parts C/D, OLS ignores entity-specific effects and temporal structure, yielding poor generalisation.
- The global model forces a single set of weights across fundamentally different data-generation mechanisms, producing suboptimal predictions everywhere.

These observations motivated our transition to **part-specific** models.

4 Part-Specific Models: Methodology and Mathematics

4.1 Part A: Ridge Regression for Correlated Linear Signals

4.1.1 Intuition and Model Selection

Part A contains i.i.d. regression rows where the response is largely explained by a linear combination of correlated covariates. When features are highly correlated, $X^\top X$ has eigenvalues near zero, and the OLS estimate $(X^\top X)^{-1} X^\top y$ becomes highly unstable: small perturbations in the data cause large swings in $\hat{w}$.

The classic remedy is **Ridge regression**, which shrinks the weight vector by adding an L_2 penalty.

4.1.2 Mathematical Formulation

Ridge regression solves:

$$\hat{w}_{\text{Ridge}} = \arg \min_w \frac{1}{2} \sum_{i=1}^n (y_i - w^\top x_i)^2 + \frac{\lambda}{2} \sum_{j=1}^d w_j^2 \quad (9)$$

where $\lambda > 0$ is the regularisation strength.

Taking the gradient with respect to w and setting it to zero:

$$\begin{aligned} \nabla_w \mathcal{L} &= -X^\top (y - Xw) + \lambda I' w = 0 \\ \Rightarrow (X^\top X + \lambda I') w &= X^\top y \\ \Rightarrow \hat{w}_{\text{Ridge}} &= (X^\top X + \lambda I')^{-1} X^\top y, \end{aligned} \quad (10)$$

where I' is the identity matrix with $I'_{00} = 0$ (we do *not* penalise the bias term, since regularising the intercept would shift all predictions away from the data mean).

4.1.3 Why Ridge and not Lasso?

- **Lasso** (L_1 penalty) induces sparsity, driving some coefficients to exactly zero. However, when features are highly correlated, Lasso tends to select one feature from a correlated group arbitrarily and discard the rest, producing unstable variable selection.
- **Ridge** shrinks all coefficients proportionally and distributes the signal across correlated features, which is preferable when all features contribute information and there is no reason to exclude any.
- Ridge has a *closed-form* solution, making it computationally efficient with no iterative optimisation required.

4.1.4 Preprocessing (Part A)

1. One-hot encode [dow, month_phase, dept, sensor_class, building_type].
2. Median-impute m1–m6.
3. Standard Z-score normalisation of continuous features.
4. Prepend bias column.

4.1.5 Hyperparameter Tuning

We searched over λ and selected the value minimising validation MAE:

$$\lambda_A^* = 95.41. \quad (11)$$

4.2 Part B: Polynomial Ridge Regression for Nonlinear Interactions

4.2.1 Intuition and Model Selection

Part B contains localised nonlinear structure: the response depends on *interactions* among features in restricted regions of feature space. A purely linear model draws a single hyperplane, which cannot capture nonlinear effects such as $y \propto x_1 \cdot x_2$ in a local region.

Rather than moving to tree-based or neural models, we extend the linear framework by explicitly constructing **polynomial features** and then applying Ridge regression in the expanded space. This is equivalent to kernel Ridge regression with a polynomial kernel, but implemented via explicit feature expansion for interpretability and control.

4.2.2 Feature Expansion $\Phi(X)$

We define a custom polynomial mapping $\Phi : \mathbb{R}^d \rightarrow \mathbb{R}^D$ comprising four components:

1. **Linear terms:** $x_1, x_2, \dots, x_d$ (d features)
2. **Quadratic interactions:** $x_i x_j$ for all $1 \leq i \leq j \leq d$ ($\binom{d+1}{2}$ features)
3. **Cubic to sextic powers:** $x_i^3, x_i^4, x_i^5, x_i^6$ for each i ($4d$ features, no cross-terms)

The total expanded dimensionality is:

$$D = d + \frac{d(d+1)}{2} + 4d. \quad (12)$$

A bias column of ones is prepended after the expansion.

Why include interactions up to degree 2 but only pure powers beyond? Full cubic or higher interactions would produce $O(d^3)$ or more features, leading to extreme multicollinearity and prohibitive memory usage. Our design captures *pairwise* interactions (the dominant source of localised nonlinearity) while adding higher-order univariate nonlinearity via x^3 – x^6 . This is a pragmatic compromise between model expressiveness and computational tractability.

4.2.3 Mathematical Formulation

After expansion, we solve the same Ridge objective in the augmented space:

$$\hat{w}_B = (\Phi(X)^\top \Phi(X) + \lambda I')^{-1} \Phi(X)^\top y. \quad (13)$$

Because the expanded feature matrix $\Phi(X)$ is vastly wider than the original, heavy regularisation is necessary to control overfitting. The selected hyperparameter reflects this:

$$\lambda_B^* = 1778.28. \quad (14)$$

4.2.4 Preprocessing (Part B)

Same as Part A (OHE, median imputation, standard normalisation), but the bias column is added *after* polynomial expansion rather than before, to avoid polluting the expansion with the constant feature.

4.3 Part C: Ridge Regression with Lag Features for Panel Forecasting

4.3.1 Intuition and Model Selection

Part C consists of panel time-series rows indexed by (`entity_id`, `timestamp`). Crucially, the task is *not* recursive multi-step forecasting: each test row must be predicted independently using only the features already provided for that row.

The key insight is that the lag features `y_lag1` ($y_{e,t-1}$) and `y_lag7` ($y_{e,t-7}$) are *directly provided as input features* and carry the dominant signal. Including them transforms the task from pure forecasting into a supervised regression problem with strong autoregressive covariates.

We therefore apply **Ridge regression** with the lag features included as regular input features, combined with standard Z-score normalisation.

4.3.2 Mathematical Formulation

The prediction for panel row (e, t) is:

$$\hat{y}_{e,t} = w^\top [1, \ x_{e,t}, \ z_{e,t}, \ m_{e,t}, \ \text{OHE}_{e,t}, \ y_{e,t}^{\text{lag1}}, \ y_{e,t}^{\text{lag7}}]^\top, \quad (15)$$

where w is obtained via Ridge closed-form (Eq. 10).

4.3.3 Preprocessing (Part C)

1. Sort by (`entity_id`, `timestamp`).
2. One-hot encode [`dow`, `month_phase`, `dept`, `sensor_class`, `building_type`, `entity_id`, `holiday_flag`].
3. Median-impute `m1–m6`.
4. **Standard Z-score normalisation** for all continuous features, the discrete `z` features, `timestamp`, `y_lag1`, and `y_lag7`.
5. Prepend bias column.

4.3.4 Hyperparameter Tuning

$$\lambda_C^* = 255.95. \quad (16)$$

The relatively large λ accounts for the high-dimensional one-hot encoding of 90 entities.

4.4 Part D: Ridge Regression with Lag-Derived Feature Engineering

4.4.1 Intuition and Model Selection

Part D is arguably the most challenging sub-problem. It combines:

- **Distribution shift:** Test timestamps (120–179) are beyond training timestamps (0–119 via Part C), so covariates shift.
- **Outlier contamination:** The training target distribution has heavier tails, with occasional extreme values.

Our strategy for Part D is to keep the same Ridge closed-form solver used in all other parts, but to invest heavily in **feature engineering**. Rather than switching to a robust loss function (see Section 5 for why IRLS was considered but not adopted), we mitigate outlier effects through preprocessing (robust normalisation and clipping of engineered features) and enrich the model’s expressive power through 11 nonlinear lag-derived features and entity-mean target encoding.

4.4.2 Y-Lag Feature Engineering

The lag features `y_lag1` and `y_lag7` are the strongest predictors for panel data. To allow Ridge regression (a linear model) to capture *nonlinear* relationships between past target values and the current target, we engineer 11 additional features from the raw lags:

Feature Name	Formula	Rationale
y_lag_product	$\ell_1 \cdot \ell_7$	Captures multiplicative interaction between short and long lags.
y_lag1_sq	ℓ_1^2	Quadratic nonlinearity of short-term lag.
y_lag7_sq	ℓ_7^2	Quadratic nonlinearity of long-term lag.
log_y_lag1	$\ln(\ell_1 + \epsilon)$	Compresses dynamic range; stabilises large-magnitude lags.
log_y_lag7	$\ln(\ell_7 + \epsilon)$	Same for the 7-day lag.
log_y_lag_product	$\ln(\ell_1 + \epsilon) \cdot \ln(\ell_7 + \epsilon)$	Interaction in log-space.
log_y_lag_ratio	$\ln\left(\frac{ \ell_1 + \epsilon}{ \ell_7 + \epsilon}\right)$	Log-ratio captures trend direction regardless of scale.
y_lag_diff	$\ell_1 - \ell_7$	First-order difference (weekly change).
y_lag_ratio	$\ell_1 / (\ell_7 + \epsilon)$	Relative change; captures proportional growth/decay.
y_lag_abs_diff	$ \ell_1 - \ell_7 $	Magnitude of weekly change (unsigned).
y_lag_mean	$(\ell_1 + \ell_7) / 2$	Smoothed lag level; reduces noise compared to either lag alone.

Table 1: Engineered lag features for Part D. Here $\ell_1 = \text{y_lag1}$, $\ell_7 = \text{y_lag7}$, and $\epsilon = 10^{-6}$ prevents division by zero and log-of-zero. Missing lags are median-imputed before engineering.

Why engineer lag features? A plain Ridge model estimates $\hat{y} = w_0 + w_1 \ell_1 + w_7 \ell_7 + \dots$, which assumes y depends *linearly* on ℓ_1 and ℓ_7 . In practice, the target often has nonlinear dependence on past values, for example, a multiplicative seasonal pattern or a log-linear trend. By providing ℓ_1^2 , $\ell_1 \cdot \ell_7$, $\ln |\ell_1|$, etc. as explicit features, we let the linear model fit a richer family of functions of the lags without switching to a nonlinear learner.

4.4.3 Entity-Mean Target Encoding

Different entities have different baseline activity levels. We capture this with:

$$\text{entity_mean_y}_e = \frac{1}{|\mathcal{T}_e|} \sum_{t \in \mathcal{T}_e} y_{e,t}, \quad (17)$$

where $\mathcal{T}_e$ is the set of training timestamps for entity e . Unseen entities fall back to the global training mean. This acts as an entity-specific intercept, complementing the one-hot encoding of `entity_id`.

4.4.4 Mathematical Formulation

After y-lag feature engineering, entity-mean encoding, one-hot encoding, imputation, and normalisation, the final feature vector for a Part D row (e, t) is:

$$\phi_{e,t} = [1, x_{e,t}, z_{e,t}, m_{e,t}, \text{OHE}_{e,t}, \ell_1, \ell_7, \underbrace{\ell_1 \ell_7, \ell_1^2, \dots, \bar{\ell}, \bar{y}_e}_{11 \text{ engineered}}]^\top. \quad (18)$$

The weight vector is obtained via the standard Ridge closed-form:

$$\hat{w}_D = (\Phi^\top \Phi + \lambda I')^{-1} \Phi^\top y, \quad (19)$$

where $I'_{00} = 0$ (unpenalised bias), identical to the Ridge formulation used in all other parts.

4.4.5 Preprocessing (Part D)

1. Sort by (`entity_id`, `timestamp`).
2. Median-impute `y_lag1`, `y_lag7`, then compute all 11 engineered lag features (Table 1).
3. Compute `entity_mean_y` from training data (Eq. 17).
4. One-hot encode [`dow`, `month_phase`, `dept`, `sensor_class`, `building_type`, `entity_id`, `holiday_flag`].
5. Median-impute `m1`–`m6`.
6. **Robust normalisation** (MAD-based with a clip threshold of 3.5) for `x1`–`x20`, `z`-features, `m`-features, `timestamp`, `y_lag1`, and `y_lag7`.
7. **Clip and Z-score normalise** all 11 engineered lag columns to $[\mu - 3\sigma, \mu + 3\sigma]$ followed by standard normalisation. This prevents extreme values from quadratic/product features from dominating the Ridge penalty.
8. Prepend bias column.

4.4.6 Hyperparameter Tuning

$$\lambda_D^* = 1.0196. \quad (20)$$

The small λ (compared to Parts A–C) reflects the high quality of the engineered features: because the 11 lag-derived features carry strong signal, the model benefits from minimally shrinking them. With adequate feature engineering, the need for heavy regularisation is reduced.

5 Models Considered but Not Adopted

A key aspect of our methodology is understanding *why* certain approaches were rejected. This section discusses models and techniques we evaluated or considered during development.

5.1 IRLS Robust Ridge Regression (Part D)

The Huber loss $L_\delta(r)$ — quadratic for $|r| \leq \delta$, linear otherwise — is the textbook remedy for outlier-contaminated regression. We considered an **Iteratively Reweighted Least Squares (IRLS)** solver that approximates Huber-loss minimisation through a sequence of weighted Ridge problems:

$$w^{(k+1)} = (X^\top W^{(k+1)} X + \lambda I')^{-1} X^\top W^{(k+1)} y, \quad (21)$$

where $W = \text{diag}(\omega_1, \dots, \omega_n)$ and $\omega_i = \min(1, \delta/|r_i|)$.

Why we did not use it.

- **Feature engineering outperformed it.** With the 11 lag-derived engineered features (Table 1) and entity-mean encoding, standard Ridge already achieved lower validation MAE than IRLS with raw features. The engineered features capture nonlinear signal that IRLS cannot recover by merely re-weighting observations.
- **Two extra hyperparameters.** IRLS introduces the Huber threshold δ and the convergence tolerance ε , increasing the search space without compensating gains.
- **Clipping already mitigates outliers.** Our preprocessing clips engineered features to $[\mu - 3\sigma, \mu + 3\sigma]$ and applies MAD-based robust normalisation, reducing outlier influence *before* the model sees the data.

5.2 Lasso Regression (L_1 Penalty)

Lasso solves $\min_w \|y - Xw\|_2^2 + \lambda\|w\|_1$, which encourages sparsity by driving some coefficients to exactly zero.

Why we did not use it.

- **Correlated features.** In Part A, features are highly correlated. Lasso arbitrarily selects one feature from a correlated group and zeroes out the rest, producing unstable variable selection. Ridge distributes the signal across all correlated features.
- **No closed-form solution.** Lasso requires iterative solvers (coordinate descent), adding implementation complexity. Ridge has a clean closed-form $(X^\top X + \lambda I)^{-1} X^\top y$.
- **No evidence of irrelevant features.** The dataset structure, with 20 continuous covariates, 8 discrete covariates, and several categorical variables, does not suggest the presence of entirely uninformative feature groups that would benefit from being zeroed out. All feature types carry plausible signal, making sparsity-inducing regularisation unnecessary.

5.3 Elastic Net

Elastic Net combines L_1 and L_2 penalties: $\lambda_1\|w\|_1 + \lambda_2\|w\|_2^2$. It handles correlated features better than pure Lasso but still requires iterative optimisation and two penalty hyperparameters. Given that Ridge alone sufficed for our multicollinearity problem, the added complexity was unjustified.

5.4 Kernel Ridge Regression / Support Vector Regression

Kernel Ridge with an RBF kernel ($K_{ij} = \exp(-\gamma\|x_i - x_j\|^2)$) can capture arbitrary nonlinearities.

Why we did not use it.

- **$O(n^2)$ memory and $O(n^3)$ time.** The kernel matrix for even Part C ($n \approx 10,800$) would require ~ 900 MB, and solving the system costs $O(n^3)$ flops.
- **Explicit polynomial expansion was sufficient for Part B.** Our custom degree-6 expansion with pairwise interactions provides enough nonlinearity for the localised interactions observed in Part B, without the computational burden of kernel methods.

- **Nystroem/random-feature approximations** were considered but introduce additional hyperparameters (number of components, kernel bandwidth) and move away from the closed-form paradigm.

5.5 Polynomial Features for Parts C/D

We used polynomial expansion only for Part B. For Parts C/D, the lag features already carry the dominant signal, and polynomial expansion of the full feature set (including 90 entity one-hot columns) would create an unmanageably large feature matrix. Instead, we applied *targeted* feature engineering (11 lag-derived features) only to the most informative features.

5.6 Why Only numpy and pandas?

We deliberately avoided `scikit-learn` and other ML libraries to:

- Maintain full control over every preprocessing and modelling step.
- Ensure that every line of the pipeline is transparent and auditable.
- Demonstrate understanding of the underlying mathematics rather than relying on black-box API calls.

6 Ablation Study: The Impact of Lag Features

To validate our modelling strategy for the panel forecasting tasks (Parts C and D), we conducted an ablation study assessing two key decisions: the inclusion of raw lag features, and the subsequent addition of engineered lag features (Table 1) alongside entity-mean encoding.

Setup. For each part, we compared three Ridge regression configurations, keeping all other hyperparameters (λ , base preprocessing) constant:

1. **No lags:** Ridge using only contemporaneous covariates (no past target values).
2. **Raw lags:** Ridge including `y_lag1` and `y_lag7` as-is.
3. **Engineered lags:** Ridge with the raw lags, the additional 11 lag-derived features, and entity-mean target encoding.

Part	No Lags	Raw Lags	Engineered Lags	Raw $\rightarrow$ Eng Δ
C	3.678	1.152	1.147	-0.005
D	6.591	3.475	2.605	-0.870

Table 2: Ablation: step-wise effect of adding raw lags and engineered lag features. Introducing basic lags significantly improves both parts, but lag engineering only benefits Part D.

Interpretation. The necessity of raw lags: As shown in Table 2, introducing the raw `y_lag1` and `y_lag7` features results in a massive MAE reduction for both parts (from 3.678 to 1.152 for Part C, and 6.591 to 3.475 for Part D). This confirms our intuition that panel forecasting relies heavily on autoregressive signals, and that past target values are the most powerful predictors in the dataset.

The asymmetry of engineered lags: For **Part C**, moving from raw lags to engineered features reduces validation MAE by only 0.005 , a negligible gain. This indicates that the relationship between the target and the raw lags is approximately linear within Part C’s in-distribution regime. The extra 11 features add complexity without meaningful benefit, justifying our decision to omit them from the final Part C pipeline to keep the model simple and avoid unnecessary overfitting risk.

For **Part D**, however, the engineered features reduce validation MAE by an additional 0.870 (from 3.475 to 2.605) , a 25% relative improvement over the raw-lag baseline. Part D’s distribution shift and heavier-tailed noise create nonlinear dependencies between past target values and the current target (e.g., multiplicative seasonal patterns, log-linear trends) that a purely linear function of ℓ_1, ℓ_7 cannot capture. The engineered features , products, squares, logarithms, ratios , let the linear model approximate these nonlinear relationships, yielding a large gain.

This step-wise ablation perfectly justifies our design choice: baseline lag features are universally necessary, but *feature engineering is applied only where it provides substantial empirical benefit*.

7 Part-Wise Error Breakdown and Results

Table 3 summarises the validation MAE for each part using the final selected hyperparameters and models.

Part	Model	λ	Validation MAE
A	Ridge Regression	$\lambda = 95.41$	1.1578
B	Polynomial Ridge (degree 6)	$\lambda = 1778.28$	8.8788
C	Ridge + Standard Norm	$\lambda = 255.95$	1.1520
D	Ridge + Lag Engineering	$\lambda = 1.0196$	2.6050

Table 3: Part-wise model summary and validation MAE.

Scoring weights. Parts C (weight 0.35) and B (weight 0.25) dominate the final score. Our architecture particularly targets these parts: polynomial expansion for B’s nonlinearity, and lag-inclusive Ridge for C’s temporal dependence. Part D further benefits from targeted lag-derived feature engineering and entity-mean encoding.

Final submission. After hyperparameter selection, we retrain each part-specific model on the combined `train + valid` data with the same preprocessing pipeline (re-fitted on the combined set) and generate predictions for all 26 854 test rows. The submission CSV contains exactly two columns: `row_id` and `y`.

Part	Key Modelling Decisions
A	Ridge closed-form; standard Z-score normalisation; OHE for categoricals; $\lambda \approx 95$ handles multicollinearity.
B	Custom polynomial expansion (all features up to degree 6, plus all pairwise interactions); Ridge on expanded space; heavy regularisation ($\lambda \approx 1778$) controls overfitting in this massive high-dimensional space.
C	Ridge on panel data with raw lag features as inputs; standard Z-score normalisation; entity-level OHE; row-level prediction (not rollout).
D	Ridge with 11 lag-derived engineered features (products, squares, logs, ratios, differences); entity-mean target encoding; MAD-based robust normalisation + 3σ -clipping of engineered columns; light regularisation ($\lambda \approx 1.02$).

Table 4: Summary of key modelling decisions per part.

8 Discussion

8.1 Why Part-Specific Models?

The four parts arise from fundamentally different data-generation processes: i.i.d. with multicollinearity (A), i.i.d. with nonlinearity (B), temporal panel data (C), and shifted/-contaminated panel data (D). A single model must compromise across all four regimes, whereas part-specific models can adapt their feature space, preprocessing, and regularisation to each sub-problem.

8.2 Why Ridge Over All Other Models?

Our entire pipeline is built on Ridge regression. This was a deliberate choice:

1. **Closed-form solution:** $(X^\top X + \lambda I)^{-1} X^\top y$ is computed in a single linear solve, with no learning rate, batch size, or convergence monitoring.
2. **Guaranteed global optimum:** The objective is strictly convex, so the closed-form solution is the unique global minimiser.
3. **Minimal hyperparameters:** Only one scalar λ per part needs tuning.
4. **Adaptability through feature engineering:** Rather than changing the *model class* for each part, we kept the model fixed and varied the *input representation* — polynomial expansion for Part B, lag-derived features for Part D, entity-mean encoding for Part D. This cleanly separates the modelling assumption (linearity in the transformed features) from the feature design.

8.3 Strengths of the Approach

- **Closed-form solutions everywhere:** All four part models use the same `ridge_closed_form` function, making the codebase simple and auditable.
- **Interpretability:** Every component, feature expansion, normalisation choice, clipping, has a clear statistical justification.
- **Robustness via preprocessing:** MAD-based normalisation, outlier clipping, and entity-mean encoding form a layered defence against heavy-tailed noise, without requiring a non-quadratic loss.
- **No data leakage:** Preprocessing is always fitted on training data; panel causality is preserved (no future information used).
- **Self-contained:** The entire pipeline uses only `numpy` and `pandas`, with no external ML libraries.

8.4 Limitations and Potential Improvements

- **Polynomial features scale poorly:** The degree-6 expansion for Part B creates a large feature matrix. Kernel Ridge regression (e.g. with an RBF kernel) or Nystroem approximation could achieve similar nonlinearity with lower memory.
- **Entity embeddings:** One-hot encoding of 90 entities creates 90 sparse features. A learned low-dimensional embedding might generalise better, especially under distribution shift.
- **Cross-validation:** We used a single train/valid split. k -fold cross-validation would provide more reliable hyperparameter estimates, at the cost of increased computation.
- **Feature selection:** We did not perform systematic feature importance analysis. Techniques like Recursive Feature Elimination or Group Lasso could potentially identify redundant features without suffering from the instability of pure L_1 regularisation under multicollinearity.
- **Robust loss functions:** IRLS or Huber regression could complement the feature engineering approach for Part D; a blended strategy may yield further gains.

9 Conclusion

We presented a comprehensive, part-specific regression framework for the ELL 409 Course Project 1. Starting from a global OLS baseline, we identified the statistical challenges specific to each part and designed targeted solutions:

1. **Part A:** Ridge regression with L_2 regularisation ($\lambda = 95.41$) handles multicollinearity among $\mathbf{x}_1$ – $\mathbf{x}_{20}$, notably the near-perfectly correlated pair $\mathbf{x}_2$ and $\mathbf{x}_{10}$ ($|\rho| \approx 0.9998$), by stabilising the Gram matrix $X^\top X$ and shrinking unstable weights.
2. **Part B:** Explicit polynomial feature expansion with pairwise interactions and individual powers up to degree 6, followed by heavily regularised Ridge regression ($\lambda = 1778.28$), captures localised nonlinear structure.
3. **Part C:** Ridge regression incorporating lag features (`y_lag1`, `y_lag7`) as first-class inputs with standard Z-score normalisation ($\lambda = 255.95$), effectively models temporal panel dependence.
4. **Part D:** Ridge regression with 11 lag-derived engineered features, entity-mean target encoding, and robust preprocessing ($\lambda = 1.0196$) addresses distribution shift

through superior feature representation rather than robust loss optimisation.

An ablation study confirmed that engineered y-lag features provide negligible benefit for Part C (MAE improvement of only 0.01) but are critical for Part D (MAE reduction from 4.20 to 2.60), justifying their selective application. We also documented why alternative approaches , IRLS, Lasso, and kernel methods , were considered but ultimately not adopted (Section 5).

All hyperparameters were selected via a strict validation protocol on `valid.csv`, and the final models were retrained on the combined `train + valid` data before submission. The entire pipeline is implemented in a single self-contained notebook using only `pandas`, `numpy`, and basic Python, with no external ML libraries.